ELECTRONIC PAYMENT GATEWAY

Step-by-Step Deployment Guide

From GitHub Repository to Running Pipeline

Audience	Developers & DevOps engineers
Prerequisites	Git, Docker Desktop or Docker Engine
Est. Setup Time	15–30 minutes (seeder: ~5 min extra)
Target OS	macOS, Windows (WSL2), Linux
Stack	MySQL · Kafka · MinIO · Python 3.11
Date	March 14, 2026

Table of Contents

1. Prerequisites

Before cloning the repository, ensure the following tools are installed and running on your machine.

1.1 Required Software

Tool	Minimum Version	Purpose	Check Command
Git	2.30+	Clone the repository	git --version
Docker Engine	24.0+	Run all containers	docker --version
Docker Compose	2.20+	Orchestrate multi-container stack	docker compose version

ℹ️ **Note:** Docker Desktop (macOS/Windows) includes both Docker Engine and Docker Compose. On Linux, install them separately from docs.docker.com.

1.2 System Resources

Resource	Minimum	Recommended	Notes
RAM	8 GB	16 GB	Kafka + MySQL + Airflow are memory-intensive
Disk	10 GB	20 GB	Docker images + MySQL data + MinIO Parquet files
CPU	2 cores	4 cores	Data seeder uses all available cores

1.3 Required Ports

The following host ports must be free before starting the stack:

Port	Service	UI / Purpose
3306	MySQL	Direct database connections
8081	Adminer	Web SQL browser — http://localhost:8081
9092	Kafka	Kafka broker (internal + external access)
9000	Kafdrop	Kafka topic browser — http://localhost:9000
9001	MinIO Console	Object storage UI — http://localhost:9001
9002	MinIO API	S3-compatible API endpoint
8082	Airflow	DAG orchestration — http://localhost:8082
2181	Zookeeper	Kafka metadata coordination

⚠️ **Warning:** If any of these ports are in use, the affected container will fail to start. Stop the conflicting service or change the host port mapping in docker-compose.yml before proceeding.

2. Clone & Configure

Step 1 Clone the Repository

Open a terminal and run the following commands to clone the repository and navigate into the project directory:

```
# Clone the repository
git clone https://github.com/YOUR_ORG/payment-gateway-pipeline.git

# Navigate into the project directory
cd payment-gateway-pipeline
```

💡 **Tip:** Replace YOUR_ORG with the actual GitHub organisation or username that owns the repository.

Step 2 Verify the Project Structure

After cloning, confirm the expected directory layout is present:

```
payment-gateway-pipeline/
├── docker-compose.yml          # Orchestrates all services
├── .env.example                # Template — copy to .env
├── Data-Sources/
│   ├── Dockerfile              # Python image for seeder/streamer
│   ├── entrypoint.sh           # MySQL readiness wait script
│   ├── generate_data.py        # Bulk historical data seeder
│   └── stream_data.py          # Real-time data streamer
├── Pipeline/
│   ├── Dockerfile.pipeline     # Python image for Kafka services
│   ├── entrypoint_pipeline.py  # Pure-Python service readiness entrypoint
│   ├── kafka_producer.py       # MySQL → Kafka extractor
│   ├── kafka_consumer.py       # Kafka → MinIO Parquet writer
│   └── wait_for.py             # TCP / schema readiness checker
└── init-scripts/
    └── init.sql                # MySQL schema + seed reference data
```

Step 3 Create the Environment File

The stack reads secrets and configuration from a .env file. Copy the provided template and fill in your values:

```
# Copy the template
cp .env.example .env

# Open in your editor
nano .env       # or: code .env  /  vim .env
```

The .env file must contain the following variables:

```
# ── MinIO ──────────────────────────────────────
MINIO_ROOT_USER=minioadmin
MINIO_ROOT_PASSWORD=minioadmin123
```

```
# ── PostgreSQL (Airflow metadata database) ────────
POSTGRES_USER=airflow
POSTGRES_PASSWORD=airflow123
POSTGRES_DB=airflow

# ── Airflow admin account ──────────────────────────
AIRFLOW_ADMIN_USER=admin
AIRFLOW_ADMIN_FIRSTNAME=Admin
AIRFLOW_ADMIN_LASTNAME=User
AIRFLOW_ADMIN_EMAIL=admin@example.com
AIRFLOW_ADMIN_PASSWORD=admin123
```

⚠️ **Warning:** Never commit the .env file to Git. The repository .gitignore should already exclude it. Verify with: git status — .env must not appear as a tracked file.

Step 4 Create Required Local Directories

Airflow needs three directories on the host machine to mount its DAGs, logs, and plugins:

```
mkdir -p dags logs plugins
```

ℹ️ **Note:** On Linux, you may also need to set the correct ownership so Airflow (uid 50000) can write logs: echo -e "AIRFLOW_UID=$(id -u)" >> .env

3. Build & Start the Stack

Step 5 Pull Base Images

Pull all Docker base images before building. This separates the slow download step from the build step and makes errors easier to diagnose:

```
docker compose pull
```

ℹ️ **Note:** This downloads MySQL, Kafka, Zookeeper, MinIO, Adminer, Kafdrop, PostgreSQL, and Airflow images. Total download size is approximately 4-6 GB depending on your cache.

Step 6 Build the Python Service Images

Build the two custom Python images — one for the data sources (seeder/streamer) and one for the Kafka pipeline (producer/consumer):

```
docker compose build
```

This builds two images:

Image	Dockerfile	Contains
data-sources image	Data-Sources/Dockerfile	mysql-connector-python, faker, tqdm
pipeline image	Pipeline/Dockerfile.pipeline	mysql-connector-python, kafka-python-ng, minio, pyarrow, faker

Step 7 Start the Full Stack

Launch all services in detached mode:

```
docker compose up -d
```

Docker Compose enforces the following startup order automatically via healthchecks:

```
1. Zookeeper        →  healthy  (cub zk-ready)
2. Kafka            →  healthy  (cub kafka-ready)
3. MySQL            →  healthy  (mysqladmin ping + init.sql)
4. data_seeder      →  runs once, exits 0
5. data_streamer    →  starts after seeder finishes
6. kafka_producer   →  starts after seeder finishes
7. MinIO            →  healthy  (HTTP /minio/health/live)
8. kafka_consumer   →  starts after Kafka + MinIO healthy
9. Airflow services →  start after PostgreSQL is ready
```

⚠️ **Warning:** The data_seeder takes 3-8 minutes to insert 155,000 transactions. The kafka_producer and data_streamer will not start until it completes. This is expected behaviour — do not restart the stack during seeding.

Step 8 Verify All Containers Are Running

Check that every container is in the running or healthy state:

```
docker compose ps
```

Expected output — all services should show "running" or "healthy":

```
NAME                    STATUS          PORTS
payment_gateway_db      running (healthy)   0.0.0.0:3306->3306/tcp
zookeeper               running (healthy)   0.0.0.0:2181->2181/tcp
kafka                   running (healthy)   0.0.0.0:9092->9092/tcp
kafdrop                 running             0.0.0.0:9000->9000/tcp
minio                   running (healthy)   0.0.0.0:9001-9002->9001/tcp
payment_adminer         running             0.0.0.0:8081->8080/tcp
payment_data_seeder     exited (0)          —
payment_data_streamer   running             —
kafka_producer          running             —
kafka_consumer          running             —
airflow-webserver       running             0.0.0.0:8082->8080/tcp
airflow-scheduler       running             —
```

💡 **Tip:** payment_data_seeder showing "exited (0)" is correct — it runs once and stops. Any other exit code indicates an error.

4. Verify the Pipeline is Working

Step 9 Check the Data Seeder Completed Successfully

```
# View seeder output — should end with "DATA GENERATION COMPLETE"
docker logs payment_data_seeder | tail -20
```

Expected last lines:

```
========================================================
  ✅  DATA GENERATION COMPLETE
========================================================
  Merchants          : 200
  POS Terminals      : 1,000
  Clients            : 10,000
  Payment Cards      : ~18,000
  Transactions       : 155,000
  Fraud Alerts       : ~31,000
  Refunds            : ~7,750
  Settlement Batches : 73,000
========================================================
```

Step 10 Check the Streamer is Running

```
docker logs payment_data_streamer | tail -5
```

Expected output — a live dashboard line updating every 5 seconds:

```
⏱  120s | 💰  240 txns (2.00/s) | 👤 0 clients | 🖥️ 0 POS | 🚨 0 campaigns | ❌ 0 err
```

Step 11 Check the Kafka Producer is Streaming

```
docker logs kafka_producer | tail -20
```

Expected output during full load phase:

```
=== PHASE 1 — FULL LOAD ===
  transactions              full load  (155000 rows) ...
  transactions              ✓ 155000 rows published → pg.transactions
  clients                   ✓ 10000 rows published  → pg.clients
  ...
=== FULL LOAD COMPLETE ===

=== PHASE 2 — INCREMENTAL POLL (every 15s) ===
```

Step 12 Check the Kafka Consumer is Writing Parquet Files

```
docker logs kafka_consumer | tail -10
```

Expected output — Parquet files being uploaded to MinIO:

```
transactions  →
s3://payment-gateway/raw/transactions/year=2025/month=03/day=14/20250314_153022_abc.parqu
et  (1000 rows, 182.4 KB, snappy)
clients       →
s3://payment-gateway/raw/clients/year=2025/month=03/day=14/20250314_153030_def.parquet
(1000 rows,  64.1 KB, snappy)
```

Step 13 Open the Web UIs

Service	URL	Credentials	What to Check
Adminer (MySQL)	http://localhost:8081	Server: mysqlUser: pg_userPass: pg_password_2024DB: payment_gateway	Browse tables — transactions should have 155,000+ rows
Kafdrop (Kafka)	http://localhost:9000	None required	Topics tab — 13 pg.* topics should appear with message counts
MinIO Console	http://localhost:9001	User: minioadminPass: minioadmin123	payment-gateway bucket → raw/ → Parquet files should be present
Airflow	http://localhost:8082	User: adminPass: admin123	DAGs page — no errors in scheduler

5. Common Operations

5.1 View Logs

```
# All services at once
docker compose logs -f

# Single service
docker compose logs -f kafka_producer
docker compose logs -f kafka_consumer
docker compose logs -f payment_data_streamer
docker compose logs -f payment_gateway_db
```

5.2 Stop the Stack

```
# Stop all containers (data is preserved in Docker volumes)
docker compose stop

# Stop and remove containers (volumes kept — data safe)
docker compose down
```

5.3 Restart a Single Service

```
# Restart just the Kafka consumer
docker compose restart kafka_consumer

# Restart producer + consumer together
docker compose restart kafka_producer kafka_consumer
```

5.4 Complete Reset — Wipe All Data

```
# WARNING: destroys ALL data in MySQL, MinIO, and Airflow
docker compose down -v

# Rebuild images and start fresh
docker compose up --build -d
```

⚠️ **Warning:** docker compose down -v permanently deletes all Docker volumes. MySQL data, MinIO Parquet files, and Airflow metadata will all be lost. Use this only to start from a clean slate.

5.5 Update After a Git Pull

```
# Pull latest code
git pull origin main

# Rebuild only the affected images and restart
docker compose up --build -d
```

5.6 Connect to MySQL Directly

```
# Interactive MySQL shell
docker exec -it payment_gateway_db \
  mysql -u pg_user -ppg_password_2024 payment_gateway

# Run a one-off query
docker exec payment_gateway_db \
  mysql -u pg_user -ppg_password_2024 payment_gateway \
  -e "SELECT COUNT(*) FROM transactions;"
```

5.7 Check Consumer Lag

```
# How far behind is the Kafka consumer?
docker exec -it kafka \
  kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --describe --group pg-minio-consumer
```

6. Troubleshooting

6.1 Diagnose Any Container

```
# Check exit code and error message
docker compose ps

# Full logs for a failing container
docker compose logs <service-name>

# Run a shell inside a container for manual inspection
docker exec -it <container-name> bash
```

6.2 Common Problems & Solutions

Symptom	Likely Cause	Fix
Container exits immediately on startup	.env file missing or a required variable is empty	Run: cat .env and verify all variables are set. Copy .env.example again if needed.
Port already in use error	Another process on the host is using the same port	Find the conflict: lsof -i :PORT (macOS/Linux) or netstat -ano | findstr PORT (Windows). Stop the conflicting process or edit the host port in docker-compose.yml.
data_seeder exits with code 1	MySQL schema not ready, or duplicate MCC in init.sql	Check logs: docker logs payment_data_seeder. If "transaction_statuses empty" — run docker compose down -v then docker compose up --build.
kafka_producer stuck at [Kafka] Attempt N/90	Kafka broker not healthy — zookeeper may not have started correctly	Check: docker compose ps zookeeper kafka. Run docker compose logs kafka. If broker crashed, try: docker compose restart zookeeper kafka.
No pg.* topics visible in Kafdrop	Producer has not completed Phase 1 full load yet	Wait for seeder to finish (exited 0). Then check: docker logs kafka_producer.
No Parquet files in MinIO	Consumer not connected or bucket not created	Check: docker logs kafka_consumer. Verify MinIO is healthy: docker compose ps minio.
Airflow webserver not reachable	airflow-init has not finished DB migration	Wait 2-3 minutes after first boot. Check: docker logs airflow-init.
Docker build fails with pip error	Network issue during image build or package version conflict	Retry: docker compose build --no-cache. If it persists, check internet connectivity from Docker.

6.3 Health Check Commands

```
# Check ZooKeeper is responding to ruok
echo ruok | docker exec -i zookeeper bash -c "cat > /dev/tcp/localhost/2181 && cat <
/dev/tcp/localhost/2181"
# Expected: imok

# Check Kafka broker is accepting connections
```

```
docker exec kafka kafka-topics --bootstrap-server localhost:9092 --list

# Check MySQL schema exists
docker exec payment_gateway_db \
  mysql -u pg_user -ppg_password_2024 payment_gateway \
  -e "SHOW TABLES;"

# Check MinIO bucket exists
docker exec minio mc ls local/payment-gateway/ 2>/dev/null || echo "Use MinIO UI at
http://localhost:9001"
```

7. Environment Variable Reference

All tuneable parameters across the stack. Variables marked Required must be set in .env. Variables marked Optional have working defaults and only need to be changed for customisation.

7.1 MinIO

Variable	Required	Default	Description
MINIO_ROOT_USER	Yes	—	MinIO admin username
MINIO_ROOT_PASSWORD	Yes	—	MinIO admin password (min 8 chars)
MINIO_BUCKET	No	payment-gateway	Bucket name where Parquet files are stored

7.2 PostgreSQL (Airflow)

Variable	Required	Default	Description
POSTGRES_USER	Yes	—	Airflow metadata DB username
POSTGRES_PASSWORD	Yes	—	Airflow metadata DB password
POSTGRES_DB	Yes	—	Airflow metadata DB name

7.3 Airflow Admin

Variable	Required	Default	Description
AIRFLOW_ADMIN_USER	Yes	—	Login username for Airflow web UI
AIRFLOW_ADMIN_PASSWORD	Yes	—	Login password for Airflow web UI
AIRFLOW_ADMIN_EMAIL	Yes	—	Admin email address
AIRFLOW_ADMIN_FIRSTNAME	Yes	—	Admin first name
AIRFLOW_ADMIN_LASTNAME	Yes	—	Admin last name

7.4 Kafka Pipeline

Variable	Required	Default	Description
POLL_INTERVAL	No	15	Seconds between CDC polls (kafka_producer)
BATCH_SIZE	No	500	Rows per Kafka send batch (kafka_producer)
FLUSH_ROWS	No	1000	Row threshold to write a Parquet file (kafka_consumer)

Variable	Required	Default	Description
FLUSH_SECONDS	No	30	Time threshold to write a Parquet file (kafka_consumer)
PARQUET_COMPRESSION	No	snappy	Parquet codec: snappy | gzip | zstd | none
KAFKA_GROUP_ID	No	pg-minio-consumer	Kafka consumer group ID

7.5 Data Generator

Variable / Flag	Default	Description
--clients	10000	Number of client records to seed
--pos	1000	Number of POS terminal records to seed
--transactions	155000	Number of historical transactions to seed
--txn-per-second	2	Live transaction rate (stream_data.py)
--clients-per-hour	2	New clients per hour (stream_data.py)
--fraud-per-hour	1	Fraud campaigns per hour (stream_data.py)
--pos-per-day	1	New POS terminals per day (stream_data.py)

8. Quick Reference Card

All commands needed for day-to-day operation in one place.

8.1 First-Time Setup (Run Once)

```
git clone https://github.com/YOUR_ORG/payment-gateway-pipeline.git
cd payment-gateway-pipeline
cp .env.example .env && nano .env          # fill in secrets
mkdir -p dags logs plugins
docker compose pull                        # download base images
docker compose build                       # build custom images
docker compose up -d                       # start everything
docker compose logs -f payment_data_seeder # watch seeding progress
```

8.2 Daily Operations

```
# Start
docker compose up -d

# Stop (data preserved)
docker compose down

# Live logs
docker compose logs -f

# Check status
docker compose ps
```

8.3 Service URLs

Service	URL	Login
MySQL (Adminer)	http://localhost:8081	pg_user / pg_password_2024
Kafka (Kafdrop)	http://localhost:9000	None required
MinIO Console	http://localhost:9001	minioadmin / <MINIO_ROOT_PASSWORD>
Airflow UI	http://localhost:8082	admin / <AIRFLOW_ADMIN_PASSWORD>

8.4 Key File Locations

What	Where
MySQL schema SQL	init-scripts/init.sql
Historical data seeder	Data-Sources/generate_data.py
Live data streamer	Data-Sources/stream_data.py
Kafka producer (MySQL → Kafka)	Pipeline/kafka_producer.py

What	Where
Kafka consumer (Kafka → MinIO Parquet)	Pipeline/kafka_consumer.py
Service readiness checker	Pipeline/entrypoint_pipeline.py
All secrets and config	.env